# CUDA 깊이 보기 1: 왜 GPU는 이런 구조가 됐는가

**Author**: 김대익
**Last Updated**: 2026-01-22
**Verified Sources**: NVIDIA CUDA Programming Guide, NVIDIA Architecture Whitepapers (Hopper, Ampere), Cornell Virtual Workshop

---

## 들어가며

CUDA를 처음 배우면 이상한 규칙들을 외우게 됩니다:

- "블록 크기는 32의 배수로"
- "메모리는 coalesced하게 접근해라"
- "branch divergence를 피해라"

왜요? "그래야 빠르니까"라는 답은 답이 아닙니다.

이 글에서는 **하드웨어 설계자의 관점**에서 GPU가 왜 이런 구조가 됐는지를 살펴봅니다. 단, 모든 설명이 NVIDIA의 공식 설명은 아닙니다. 공식 문서에서 확인된 내용과 그로부터 추론한 내용을 구분해서 표시했습니다.

---

## 1. 근본적인 제약: 실리콘, 전력, 메모리

칩을 설계할 때 세 가지 근본적인 제약이 있습니다:

**실리콘 면적**: 칩 크기에는 한계가 있습니다. 트랜지스터를 어디에 쓸지 선택해야 합니다.

**전력**: 발열 한계가 있습니다. H100의 경우 700W TDP입니다. [NVIDIA H100 Datasheet]

**메모리 대역폭**: 아무리 연산이 빨라도 데이터를 못 가져오면 소용없습니다.

CPU와 GPU는 이 제약 안에서 **다른 트레이드오프**를 선택했습니다.

---

## 2. CPU의 선택: Latency를 줄이자

CPU는 **하나의 스레드를 최대한 빠르게** 실행하는 데 집중합니다.

**분기 예측기 (Branch Predictor)**: CPU는 분기 예측기에 상당한 면적을 투자합니다. 현대 CPU의 분기 예측 정확도는 95%+ 입니다.

**Out-of-Order 실행**: 명령어 간 의존성을 분석하고, 실행 가능한 것부터 실행합니다.

**큰 캐시**: 메모리 접근 latency를 줄이기 위해 L1, L2, L3 캐시에 칩 면적의 상당 부분을 씁니다.

---

## 3. GPU의 선택: Latency를 숨기자

GPU는 **완전히 다른 접근**을 합니다.

> "Latency를 줄이는 건 비싸다. 대신 **숨기자**."

어떻게? **스레드를 엄청나게 많이 띄워서**, 한 스레드가 기다리는 동안 다른 스레드를 실행합니다.

**[공식 확인된 사실]** NVIDIA CUDA Programming Guide에 따르면:

- GPU는 warp 단위로 스레드를 스케줄링
- 한 warp이 메모리를 기다리는 동안 다른 warp을 실행
- 이를 통해 메모리 latency를 숨김

---

# 4. SIMT와 Warp: 왜 32인가?

## 4.1 공식 문서에서 확인된 사실

NVIDIA CUDA Programming Guide [1]에서:

> "Within a thread block, threads are organized into groups of 32 threads called warps. A warp executes the kernel code in a Single-Instruction Multiple-Threads (SIMT) paradigm."

Cornell Virtual Workshop [2]에서:

> "The reason for bundling threads into warps of 32 is simply that in NVIDIA's hardware, CUDA cores are divided into fixed groups of 32."

## 4.2 SIMT vs SIMD

NVIDIA Technical Blog [3]에서:

> "NVIDIA GPUs execute warps of 32 parallel threads using SIMT, which enables each thread to access its own registers, to load and store from divergent addresses, and to follow divergent control flow paths."

SIMT의 특징:

- 각 스레드가 자신만의 레지스터를 가짐
- 각 스레드가 다른 주소에서 load/store 가능
- 각 스레드가 다른 control flow path를 따를 수 있음 (divergence)

## 4.3 왜 하필 32인가? [추론]

⚠ **주의**: NVIDIA가 "왜 32인가"에 대해 공식적으로 설명한 문서는 없습니다. 아래는 하드웨어 특성에 기반한 추론입니다.

**가능한 이유들:**

1. **메모리 트랜잭션 크기**: GPU 메모리 트랜잭션은 보통 128 bytes. 32 threads × 4 bytes (float) = 128 bytes. 하나의 warp이 coalesced access하면 하나의 트랜잭션으로 처리 가능.

2. **명령어 fetch 효율**: 32개 스레드가 명령어를 공유하면 fetch 오버헤드가 1/32.

3. **하드웨어 구현**: Cornell에서 언급한 대로 "CUDA cores are divided into fixed groups of 32".

4. **역사적 이유**: 초기 CUDA(Compute Capability 1.0)부터 warp size는 32였고, 호환성을 위해 유지되고 있을 수 있음.

---

# 5. 메모리 시스템: Coalescing의 물리적 이유

## 5.1 공식 문서에서 확인된 사실

CUDA Programming Guide의 Memory Coalescing 섹션에서:

- Global memory 접근은 32-, 64-, 또는 128-byte 트랜잭션으로 처리됨
- Warp의 스레드들이 연속된 주소에 접근하면 적은 트랜잭션으로 합쳐짐
- 흩어진 주소에 접근하면 여러 트랜잭션이 필요

## 5.2 DRAM의 물리적 특성 [일반적 사실]

이것은 NVIDIA 특정이 아닌 DRAM 일반의 특성입니다:

**DRAM 읽기 과정**:

1. Row 활성화 (Row를 "Row Buffer"로 복사) - **느림**
2. Column 선택 (Row Buffer에서 원하는 Column 읽기) - **빠름**
3. 같은 Row의 다른 Column 읽기 - **빠름**

**Burst Mode**: DRAM은 연속된 데이터를 한꺼번에 전송하도록 설계되어 있습니다.

> **이것이 Memory Coalescing이 중요한 물리적 이유입니다.**

---

# 6. 레지스터 파일: Zero-overhead Context Switching

## 6.1 공식 스펙

**H100 GPU** [NVIDIA H100 Whitepaper]:

- SM당 65,536 registers (블록당)
- SM당 최대 2048 threads
- SM당 최대 64 warps

**A100 GPU** [NVIDIA A100 Whitepaper]:

- SM당 65,536 registers
- SM당 64 CUDA cores (FP32)

## 6.2 왜 레지스터가 그렇게 많은가?

**[공식 확인된 개념]** NVIDIA는 이를 "zero-overhead scheduling"이라고 부릅니다:

> GPU는 active warp들의 레지스터를 모두 칩에 유지합니다. Warp 전환 시 레지스터를 저장/복원할 필요가 없어서 0 cycle에 전환 가능합니다.

**계산 예시** (H100):

- 2048 스레드가 상주 가능
- 각 스레드가 32개 레지스터 사용 시: 2048 × 32 = 65,536 레지스터 필요

---

# 7. Shared Memory와 L1 Cache

## 7.1 공식 스펙

**H100 GPU** [NVIDIA Hopper Architecture Whitepaper]:

- SM당 256KB combined L1/Shared Memory
- 프로그래머가 carveout 비율 설정 가능

**A100 GPU**:

- SM당 192KB combined L1/Shared Memory

## 7.2 왜 Shared Memory가 따로 있는가?

**[공식 설명]** CUDA Programming Guide에서:

Shared Memory의 특징:

- 프로그래머가 명시적으로 데이터 배치
- 같은 thread block 내 스레드들이 공유
- L1 cache보다 예측 가능한 성능

**Bank 구조** [공식]:

- 32 banks
- 각 bank는 4 bytes (32 bits) 폭
- 같은 bank의 다른 주소에 동시 접근 시 **bank conflict** 발생

---

# 8. H100 GPU 실제 스펙 요약

**[공식 스펙]** NVIDIA H100 Tensor Core GPU Architecture Whitepaper:

| 항목 | H100 SXM5 |
|---|---|
| SMs | 132 |
| CUDA Cores (FP32) | 16,896 (128/SM × 132) |
| Tensor Cores | 528 (4/SM × 132) |
| Registers per SM | 65,536 |
| Shared Memory per SM | 최대 228KB |
| L2 Cache | 50MB |
| Memory Bandwidth | 3 TB/s (HBM3) |
| TDP | 700W |

## 9. 설계 철학 정리

| 설계 결정 | CPU | GPU |
|---|---|---|
| Latency 처리 | 줄인다 (캐시, 예측) | 숨긴다 (많은 스레드) |
| 명령어 fetch | 코어별 독립 | Warp 공유 [공식] |
| 분기 처리 | 예측 + 투기적 실행 | Divergence 감수 [공식] |
| 레지스터 | 적고 빠름 | 많음 (zero-overhead switching) [공식] |

## 10. 마치며

GPU가 이렇게 생긴 이유를 알면, 최적화 규칙들이 더 이상 "외워야 하는 것"이 아니게 됩니다.

- **Coalescing**: DRAM의 물리적 특성과 GPU 메모리 트랜잭션 크기 때문
- **32 스레드**: 하드웨어가 32개 단위로 묶여있음 (구체적 이유는 비공개)
- **많은 스레드**: Latency를 줄이는 대신 숨기는 전략
- **Shared Memory**: 예측 가능한 접근 패턴을 프로그래머가 명시적으로 관리

다음 글에서는 **실제 코드를 튜닝하면서 어디서 막히고 어떻게 해결하는지**, 프로파일링과 함께 살펴보겠습니다.

## References

1. NVIDIA CUDA C++ Programming Guide (https://docs.nvidia.com/cuda/cuda-programming-guide/)
2. Cornell Virtual Workshop - GPU Architecture (https://cvw.cac.cornell.edu/gpu-architecture/)
3. NVIDIA Technical Blog - Using CUDA Warp-Level Primitives (https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/)
4. NVIDIA H100 Tensor Core GPU Architecture Whitepaper
5. NVIDIA Hopper Architecture In-Depth (https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/)

## 변경 이력

- 2026-01-22: 초판 작성. 공식 문서 기반으로 검증, 추론 부분 명시